Implementation walkthrough

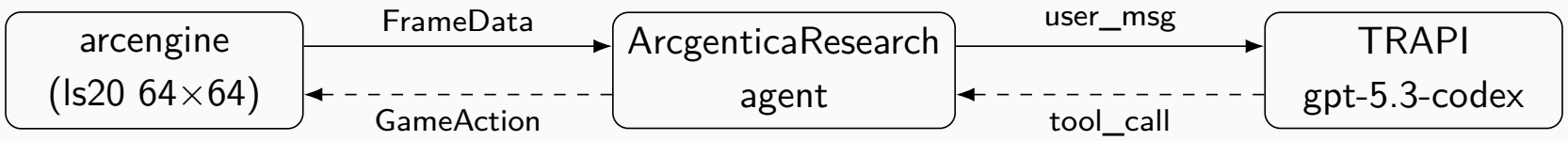
Symbolica agent + WorldCoder + DreamCoder + MCTS + Wake-Sleep

ARC-AGI-3 Research

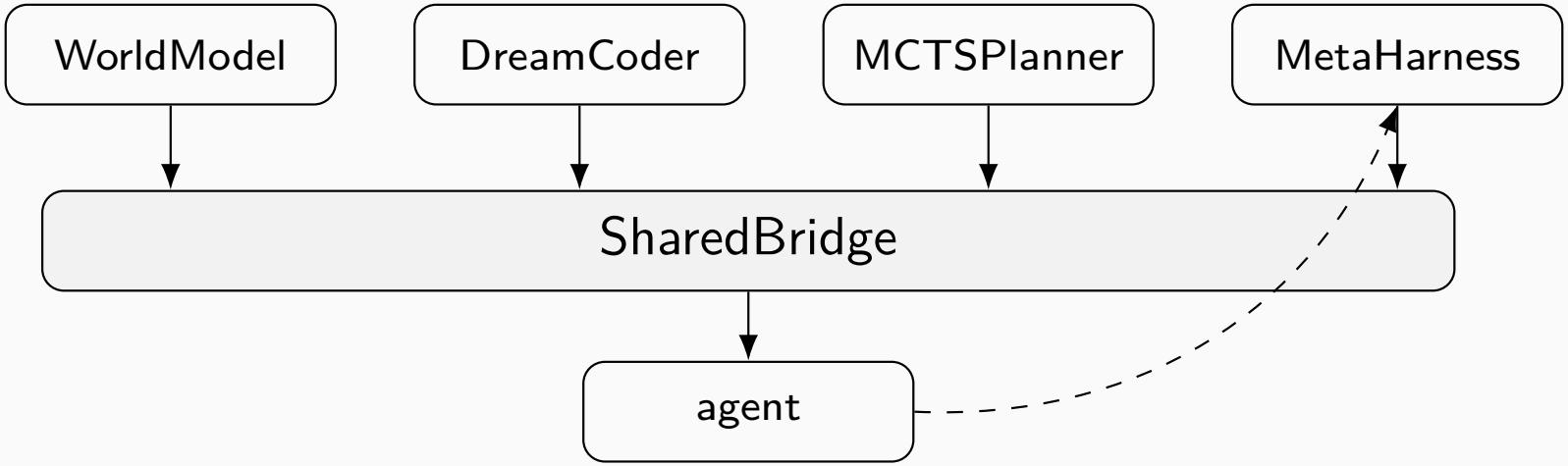
2026-04-17

Part 1: Overview

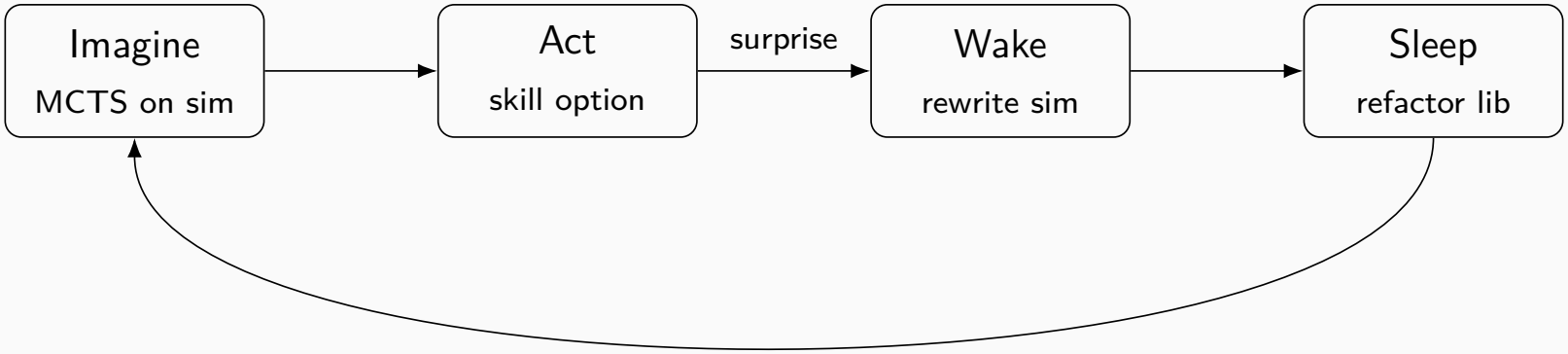
Symbolica agent on top of TRAPI



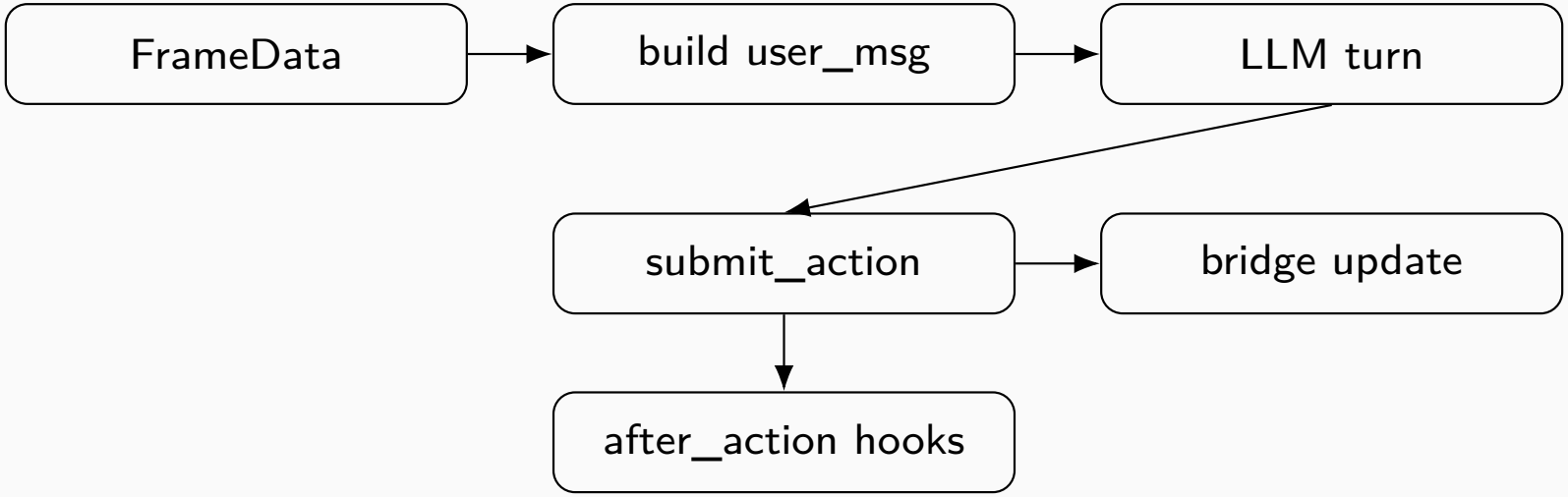
Four modules on a shared bridge



Wake-sleep loop



Per-turn data flow



Part 2: Prompts and functions

Agent tools exposed to the LLM

```
history(n, wins_only)
memories_{summaries, get, add, query}
frame_render(keys, gap, y_ticks, x_ticks, crop)
frame_diff(margin)          frame_change_summary(margin)
frame_render_diff(keys, gap, crop='auto')
frame_find(values)          frame_bounding_box(values)
frame_color_counts()
submit_action(action_name, predict?, world_update?,
              propose_skill?, env_note?)
```

Per-turn user_msg skeleton

```
Game ls20 | Level 0/6 | budget 67/160
State NOT_FINISHED | Available {1,2,3,4,RESET}

<64x64 grid via Frame.render with y_ticks+x_ticks>

[Wake]        last surprise summary
[Observed-Law Details]  per-action mean_diff_cells
[Skill Library]         top-8 by score
[Persistent Notes]      balanced memory slice
[Exploration Phase?]    if still sampling
```

WorldCoder contract (verbatim)

```
REAL WORLD MODEL contract:
  def predict_effect(action, observation) -> dict
  observation has 'grid' (64x64) and 'per_action_delta_hints'.
MANDATORY fields:
  expect_change: True | False
  expected_diff_band: 'zero' | 'small' | 'large'
  expected_diff_cells: int
  expected_next_grid: List[List[int]]     # full next grid
ANTI-CHEAT:
  'unknown' or None               = WRONG
  expected_next_grid == input     = WRONG when diff_band != zero
  delta-only submission           = WRONG
```

WorldCoder template (verbatim)

```
def predict_effect(action, observation):
    g = observation['grid']; H, W = len(g), len(g[0])
    new = [row[:] for row in g]
    hints = observation.get('per_action_delta_hints', {}).get(action, {})
    for change_list in hints.get('sample_changes', [])[:1]:
        for x, y, v in change_list:
            if 0 <= y < H and 0 <= x < W: new[y][x] = v
    n = sum(1 for y in range(H) for x in range(W) if g[y][x] != new[y][x])
    return {'expect_change': n > 0,
            'expected_diff_band': 'zero' if n==0 else ('small' if n<=4 else 'large'),
            'expected_diff_cells': n,
            'expected_next_grid': new}
```

DreamCoder: five skill kinds

kind	source	role
`abstract_primitive`	`seed_library.py`	10 seeds (BFS, button semantics)
`observed_law`	`consolidate_experience`	per-action empirical stats
`mcts_proposal`	`propose_from_mcts`	MCTS top path
`wrapper`	`sleep_refactor`	N-gram prefix merge
(free)	agent `propose_skill`	free-form

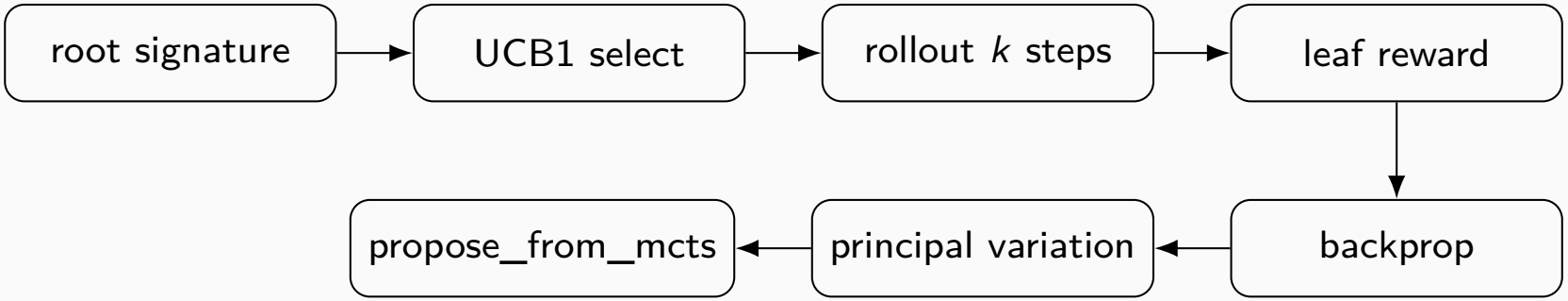
DreamCoder methods

```
commit_skill(record)          # begin option execution
next_skill_action()           # returns body[i], LLM bypassed
on_skill_step(pred, obs)      # surprise -> abort + queue wake
sleep_refactor()              # N-gram wrappers + demote
consolidate_experience(wm)    # emit observed_law every 10 steps
```

MCTS planner

```
class MCTSPlanner:  # modules/planner.py
    max_depth = 4;  n_simulations = 64;  ucb_c = 1.4
    def plan(world_model, root_signature, available_actions):
        if wm._best_draft().transition_accuracy < 0.4:  # H5b gate
            return {'gate_ok': False}
        # UCB1 tree; each rollout calls wm.simulate_step (pure Python)
        reward = 3*new_family + 2*branch_escape + 1*accuracy - 1*repeat
        return {'mcts_top_paths': [...],
                'bfs_top_paths':  [...],  # length-matched baseline
                'advantage': mcts_top_reward - bfs_mean_reward}
```

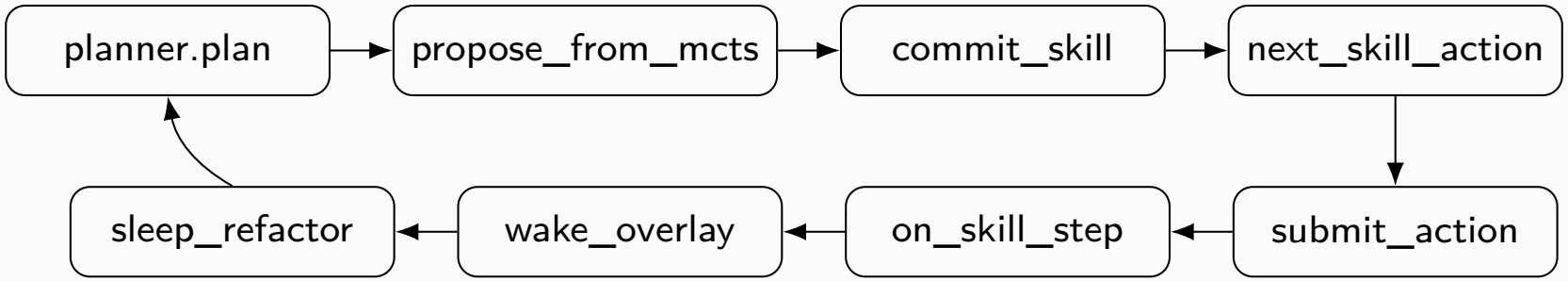
MCTS rollout → reward → skill



Reward signal for a simulated transition

```
_reward_from_sim(sim, family_chain):
  r  = 3 * sim.new_family
  r += 2 * sim.branch_escape
  r += 1 * sim.transition_accuracy
  r += 2 * I[sim.next_sig_hint matches 'new/unseen']
  r -= 1 * family_chain.count(sim.next_sig)   # repeat penalty
  r -= 2 * sim.exception_flag
  return r
```

Commit flow: imagine → act → wake → sleep



Part 3: Response evolution

Side-channel fields returned by the LLM

```
submit_action(
    action_name = "ACTION3",
    predict     = {"expect_change": True, "band": "large"},
    world_update= "‘‘‘python\ndef predict_effect(...): ...\n‘‘‘",
    propose_skill = {"name": ..., "body": [...], "kind": ...},
    env_note    = "short observation"
)
```

Bridge channels written per turn

```
bridge.current_prediction      <- predict
bridge.proposed_skills  .append(propose_skill)
bridge.surprises        .append(SurpriseEvent if mismatch)
bridge.pending_wake_triggers.append(event)
bridge.pending_mcts_proposals.append(...)  # from planner
bridge.current_skill_in_execution = SkillExecutionState
bridge.scientist_state.update(...)
bridge.shared_hints["last_synthetic_prediction"] = ...
```

Scorer: _unit_test_matches

```
def _unit_test_matches(pred, test):
  # 1 expect_change : must be concrete True|False and correct
  # 2 expected_diff_band : must be 'zero'|'small'|'large' and correct
  # 3 observation_prediction : if set, must agree
  # 4 progress_prediction    : if set, must agree
  # 5 next_signature_hint    : semantic / exact hash check
  # 6 expected_diff_cells    : within +/- 30%
  # 7 IDENTITY GUARD : pred.grid == input and test.diff != zero -> FAIL
  # 8 FULL GRID      : if test has after_grid, >=95% cell match required
  return all_gates_pass
```

Draft score formula

```
WorldDraft.score():
    self_score
  + 1.50 * empirical_matches       - 2.00 * empirical_mismatches
  + 1.75 * simulator_matches       - 2.00 * simulator_mismatches
  + 2.00 * transition_accuracy
  + 0.25 * revisions
  - 0.50 * silence_penalty   (when <50% of tests scored)
```

Example: wildcard response

```
run: arcgentica_v3_6_sym_prompt_20260416_2338

predict_effect returned:
  {'expect_change': None, 'expected_diff_band': 'unknown'}

matcher under v3.6 (soft):  64/64 pass   -> transition_accuracy 1.00
matcher under v3.15 (strict):  0/64 pass -> gate 1 fails on None
```

Example: 1-cell delta response

```
run: arcgentica_v3_11_strict_penalty_20260417_0234

predict_effect returned:
  {'expect_change': True, 'band': 'large',
   'expected_diff_cells': 1,
   'expected_grid_delta': [[32, 32, 9]]}

scorer: delta-only submission -> FAIL (v3.12+ requires full grid)
```

Example: identity response

```
run: arcgentica_v3_14_wake_fix_20260417_0356

predict_effect returned:
  {'expect_change': False, 'expected_next_grid': observation['grid']}

scorer under v3.15+ identity guard:
  pred.grid == input and test.diff == 'large' -> FAIL
```

Example: v3.15.1 current response

```
run: arcgentica_v3_15_1_builtins_20260417_0430

predict_effect draft focus:
  "terminal-inert short-circuit precedence"

[Wake]      memories  104   (v3.14 had 0)
unit_tests passed / seen  1 / 64    transition_accuracy 0.016
structured predictions                53 / 55
```

Artifact paths

```
code
  research_extensions/modules/
    world_model.py dreamcoder.py planner.py
    meta_harness.py seed_library.py
  research_extensions/bridge.py  registry.py
  agents/templates/arcgentica_research/agent.py
per-run
  research_logs/shared/ls20-cb3b57cc/<ns>/
    world_model.json  dreamcoder_library.json
    meta_harness.json  planner_state.json  memories.json
viz & talk
  presentation/bestiary.html
  presentation/trace_viewer_<ns>.html
  reports/v3_mcts_wake_sleep_talk.pdf
```